

Zamanlama Problemi Olarak Teknoloji Takibi

Ömer Faruk Yavuz

August 2026

Özet

Teknoloji takibi bir bilgi biriktirme değil zamanlama işidir. Ürettiği asıl çıktı bilginin kendisi değil, gerektiğinde kullanılabilecek bir seçenektir; bir teknoloji yaygınlaştıktan sonra öğrenildiğinde bilgi doğru kalır ama karar değeri kalmaz. Erken benimseme ile geç kalmanın maliyetleri asimetriktir: birincisi görünür ve bütçelenebilir, ikincisi dağınık ve gecikmelidir. Kurumların sistematik olarak geç kalmasının nedeni bu asimetridir.

İzlenmesi gereken şey teknolojilerin kendisi değil, altlarındaki kısıtlardır. Her yeni araç, bir kısıtın kaydığı anda açılan boşluğun doldurulmasıdır; kısıtı gözleyen boşluğun nerede açılacağını kestirebilir. Basitleştirme iddiasındaki araçlar karmaşıklığı ortadan kaldırmaz, başka bir konuma taşır; değerlendirmede sorulacak soru “ne kolaylaştı” değil “karmaşıklık nereye gitti” olmalıdır.

Soyutlama, ayrıntının bir bölümünü gizleyip yerine daha küçük bir arayüz koyan bir *çıkarma* işlemidir. Bir soyutlamanın yeterliliği doğruluğuyla değil, kesim yerinin isabetiyle ölçülür; uygun kesim yeri tasarım anında türetilemez, çalışan sistemlerden damıtılır. Aynı belirsizlik fiziksel tasarımda da geçerlidir: bir bileşenin gerekli olup olmadığı, çıkarılmadan önce güvenilir biçimde bilinmez. Her iki alanda da fazlalık varsayılan durumdur, çünkü ekleme kararını veren ile bedelini ödeyen aynı taraf değildir.

Giriş

Bir teknolojik gelişme hakkında bilgi edinmenin iki ayrı biçimi vardır. Birincisinde okuyucu birbirinden bağımsız olayların listesini biriktirir: bir yazılımın yeni sürümü çıkmıştır, bir şirket bir aracı açık kaynak yapmıştır, bir standart yayımlanmıştır. İkincisinde okuyucu, bu olayların yerleştirileceği bir yapı kurar ve yapının kendisinden öngörü türetir.

Model terimi bu metinde ikinci durumdaki yapıyı adlandırmak için kullanılmaktadır. Bir modele sahip olmak iki şey demektir: yeni duyulan bir gelişmenin konulacağı hazır bir yerin bulunması, ve o yere konulduğunda henüz gerçekleşmemiş bir şey hakkında tahmin üretilebilmesi.

Soyutlama terimi ise bir sistemin ayrıntılarının bir bölümünü gizleyerek yerine daha küçük bir arayüz koyan işlemi adlandırır. İşletim sistemlerindeki dosya kavramı bunun standart örneğidir: dosyanın altında sektörler, blok tahsisi ve disk geometrisi bulunurken kullanıcıya dört çağrılık bir arayüz sunulur.

Bu iki kavram ilk bakışta farklı alanlara aittir; biri bilgi edinme pratiği, diğeri sistem tasarımı tekniğidir. Aralarındaki bağ şudur: her ikisi de neyin atılabileceğine, neyin tutulması gerektiğine dair bir karar problemidir ve her ikisinde de karar, sonucu görülmeden verilmek zorundadır.

Yaygın bir kabul, teknoloji takibini bir bilgi miktarı sorunu olarak görür: ne kadar çok izlenirse o kadar iyidir. Bu metin bu kabulden ayrılmakta, takibi bir zamanlama ve seçim problemi olarak ele almaktadır. Benzer biçimde, soyutlama tasarımı hakkındaki yaygın kabul, yeterince özenli bir tasarımın kusursuz bir arayüz üretebileceğidir; burada savunulan görüş, her soyutlamanın kasıtlı bir eksiltme olduğu ve kusursuzluğun değil dayanıklılığın ölçüt alınması gerektiğidir.

Metin, bilgi edinme ve tasarım kararlarının yapısıyla sınırlıdır. Belirli bir teknolojinin değerlendirilmesi, teknoloji seçimi için bir karar prosedürü ya da kurumsal benimseme süreçlerinin yönetimi kapsam dışındadır.

Teknoloji Takibinin Yöntemi

Değer ile türev ayrımı

Tekil bir sürüm bilgisi bir *değer*dir. Ardışık sürümler arasındaki yönelim ise bir *türev*dir.

Bir ilişkisel veritabanı sisteminin son beş sürümünde çoğaltma (*replication*), mantıksal çözümleme (*logical decoding*) ve yedekleme başlıklarının öne çıkması, geliştirici topluluğunun dağıtık kullanım senaryosunu öncelediğini gösterir. Bu bilgi tek bir sürüm notunda bulunmaz; ancak beş sürümün karşılaştırılmasıyla ortaya çıkar.

Ayrımın önemi şuradadır: değer yalnızca bir durumu bildirir, yön bilgisi taşımaz. Türev ise yön bilgisi taşır ve yönden bir sonraki adım kestirilebilir. Türevi gözleyen okuyucu bir sonraki sürümün hangi alana yöneleceğini tahmin edebilir; yalnızca değeri gözleyen okuyucu sürüm numarasının arttığını bilir.

Kaynakların el sırası

Enformasyon kaynakları, olayın kendisine olan yakınlıklarına göre sıralanabilir. Sosyal medya paylaşımları üçüncü el, ikincil çözümleme yazıları ikinci el kaynaklardır. Birinci el kaynaklar şunlardır: sürüm notları, değişiklik kayıtları (*changelog*), RFC belgeleri¹, teknik şartnameler, olay sonrası çözümleme raporları (*postmortem*), sorun izleme sistemlerindeki tartışmalar ve sürüm kontrol sistemindeki işleme (*commit*) mesajları.

Sıralamanın gerekçesi şudur: bir özelliğin *neden* eklendiği bilgisi duyuru metninde bulunmaz. Duyuru metni tanıtım işlevi gördüğü için sonucu aktarır, gerekçeyi aktarmaz; gerekçenin aktarılması çoğu durumda kurumun çıkarına da değildir, çünkü gerekçe genellikle bir eksikliği ya da bir dış baskıyı içerir. Gerekçe, tasarım tartışmasının yapıldığı yerde kayıtlı kalır.

Doğrudan deneyim ölçütü

Bir araç hakkında kanaat sahibi olmak onu çalıştırmayı gerektirir. Çalıştırmadan edinilen kanaat, başkasının kanaatinin taşınmasından ibarettir ve taşıyan kişi o kanaatin hangi koşullar altında oluştuğunu bilmez.

Tanıtım düzeyindeki bir deneme de yetersiz kalır, çünkü yalnızca tanıtım metninin doğruluğunu sınar. Tanıtım örnekleri, aracın en iyi çalıştığı senaryoyu göstermek üzere seçilmiştir.

İşlevsel ölçüt şudur: kullanıcının kendi gerçek problemlerinden biri o araçla bir kez çözülmelidir. Karşılaşılan ilk yapısal engel, aracın gerçek karakterini gösterir. Bu engel tanıtım metninde yer almaz, çünkü metni yazan taraf onu gizlemekte çıkar sahibidir.

¹RFC (*Request for Comments*), internet standartlarının taslak ve nihai hâllerleriyle yayımlandığı belge dizisidir.

Takasın belirlenmesi

Her teknoloji bir kısıtı gevşetir ve karşılığında bir bedel talep eder. Kapsayıcı (*container*) teknolojisi bağımlılık yönetimi sorununu büyük ölçüde çözmüş, karşılığında imaj boyutu ve katman yönetimi karmaşıklığını getirmiştir. Konteyner düzenleme sistemleri dağıtım sorununu çözmüş, karşılığında sürekli bakım gerektiren bir yapılandırma yükü getirmiştir.

Bu nedenle “X iyi midir” sorusu yanlış kurulmuş bir sorudur; bağlamdan bağımsız bir cevabı yoktur. Doğru kurulmuş biçimi şudur: X neyi neye takas etmektedir ve bu takas mevcut koşullarda kabul edilebilir midir. İkinci soru cevaplanabilir, birincisi değildir.

Başarısız olmuş yaklaşımların bilgisi

Bir yaklaşımın hangi noktaya kadar yayıldığı kadar nerede durduğu da bilgidir. Bir teknolojinin yayılma eğrisinin neresinde bulunduğunu kestirebilmek, benzer eğrilerin daha önce nasıl seyrettiğini bilmeyi gerektirir.

Yeniliklerin toplumsal yayılımının aşamalı ve büyük ölçüde öngörülebilir bir örüntü izlediği, yayılım çalışmalarının kurucu metninde gösterilmiştir (Rogers, 1962). Bu örüntüyü bilen okuyucu, bir yaklaşımın erken benimseyenler aşamasında mı yoksa yaygınlaşma aşamasında mı bulunduğunu ayırt edebilir.

Tarihsel derinlik

Yeni görünen çözümlerin önemli bir bölümü, daha önce başka koşullar altında denenmiş fikirlerin yeniden ortaya çıkışıdır. Sunucusuz hesaplama (*serverless*), ana bilgisayar döneminin zaman paylaşımı modelinin bir türevidir: kullanıcı makineyi değil, makinede geçirdiği süreyi satın alır. Mikroservis mimarisi, hizmet yönelimli mimarinin (SOA) daha küçük ölçekli ve daha hafif protokollü biçimidir.

Bu bağı kurabilen okuyucu yeni yaklaşımı hızla yerleştirir, çünkü yapının hangi sorunları çözdüğünü ve hangilerini çözmediğini önceki örnekten bilmektedir. Bağ kuramayan okuyucu aynı öğrenme sürecini baştan yaşar ve önceki kuşağın yaptığı hataları tekrar eder.

Süzme ölçütleri

Bir gelişmenin kalıcı olup olmadığını belirlemek için dört soru işlevseldir:

- Gelişme altı ay sonra hâlâ tartışılmakta mıdır.
- Üretim ortamında kimler tarafından ve hangi ölçekte kullanılmaktadır.
- Geliştirmeyi kim finanse etmektedir ve hangi çıkar nedeniyle.
- Anlatılan başarı örneklerinde gerçek ölçek var mıdır, yoksa gösterim düzeyinde mi kalmaktadır.

Bu dört soruyu geçemeyen içerik haber niteliğindedir ve modele katkı sağlamaz.

Kapsamın sınırlandırılması

Bütün alanların aynı derinlikte izlenmesi mümkün değildir; deneyen kişi hiçbirini yeterince izleyemez. Üç halkalı bir ayırım işlevseldir.

Derin halka, kullanıcının fiilen çalıştığı üç-dört teknolojiden oluşur ve kaynak kodu düzeyinde izlenir. **Orta halka**, komşu alanlardır; ne yaptıklarının ve hangi koşulda uygun olduklarının bilinmesi yeterlidir, kullanma becerisi gerekmez. **Sığ halka**, yalnızca varlığından haberdar olunan alanlardır; ihtiyaç doğduğunda nereye bakılacağına bilinmesi yeterlidir.

Takibin Yapısal Mantığı

Zamanlama problemi

Bir teknoloji yaygınlaştıktan sonra öğrenildiğinde edinilen bilgi doğrudur, ancak karar değeri taşımaz; verilecek bir karar kalmamıştır.

Takibin ürettiği asıl çıktı bilgi değil **opsiyondur**: henüz kullanılmayan, ancak gerektiğinde kullanılacak bir seçenek. Opsiyonun değeri kullanılmasından değil, elde bulunmasından doğar. Bu nedenle takibin ölçüsü öğrenilen şeyin miktarı değil, öğrenmenin karara göre zamanlamasıdır. Zamanında edinilmiş sınırlı bilgi, geç edinilmiş kapsamlı bilgiden değerlidir.

Maliyet asimetrisi

Erken benimsemenin maliyeti görünür ve kısa vadelidir: olgunlaşmamış araç, eksik ekosistem, yetersiz belgelendirme ve çoğunlukla bir yeniden yazım. Bu maliyetler ölçülebilir, bütçelenebilir ve belirli bir karara bağlanabilir.

Geç kalmanın maliyeti fatura biçiminde gelmez. Mimarının bir kuşak geride kalması, nitelikli çalışan bulmakta zorlanması ve rakibin aynı işi çok daha kısa sürede yapabilir hâle gelmesi biçiminde birikir. Hiçbiri tek bir kaleme yazılmaz ve hiçbirisi belirli bir karara atfedilemez.

Kurumların sistematik olarak geç kalmasının nedeni bu asimetridir. Erken benimsemenin acısı doğrudan ve kişiselleşebilir biçimde hissedilir; kararı veren kişi sonuçtan sorumlu tutulur. Geç kalmanın acısı dağıtık ve gecikmelidir; kimse sorumlu tutulmaz. Karar vericiler görünür maliyetten kaçınırken görünmez maliyeti biriktirirler.

İzlenmesi gereken şey kısıtlardır

Teknolojiler hızlı, kısıtlar yavaş değişir. Depolama maliyeti düşmüş, tek çekirdek başına işlem hızındaki artış durmuş, hesaplama birimi başına maliyet önemli ölçüde gerilemiştir; buna karşılık insanlar arası koordinasyon maliyeti pahalı kalmayı sürdürmektedir.

Her yeni teknoloji, kısıtlardan birinin kaydığı anda açılan boşluğun doldurulmasıdır. Grafik işlem birimi başına maliyetin düşmesi, önceden ekonomik olmayan hesaplama yoğun yaklaşımları uygulanabilir kılmıştır; ortaya çıkan araçlar bu kaymanın sonucudur, nedeni değil.

Kısıtı gözleyen okuyucu boşluğun nerede açılacağını kestirebilir. Yalnızca ürünü gözleyen okuyucu ise birbirini açıklamayan sonsuz sayıda bağımsız olay izlemiş olur.

Karmaşıklığın korunumu

Basitleştirme iddiasıyla ortaya çıkan bir araç, ortadan kaldırdığını öne sürdüğü karmaşıklığı çoğunlukla başka bir yere taşımıştır.

Nesne-ilişkisel eşleme katmanı (ORM)² sorgu dili yazma zorunluluğunu kaldırır; karmaşıklık, üretilen sorguların başarımının denetlenmesi alanına geçer. Çöp toplayıcı (*garbage collector*)³ bellek yönetimini programcıdan alır; karmaşıklık, duraklama sürelerinin öngörülmesi alanına geçer.

Bir aracın değerlendirilmesinde sorulacak soru “ne kolaylaştı” değil “karmaşıklık nereye taşındı” olmalıdır. İkinci sorunun cevabı bulunamıyorsa aracın davranışı henüz anlaşılmamış demektir.

Bu ilke kullanıcı arayüzü tasarımı bağlamında *karmaşıklığın korunumu yasası* adıyla anılmakta ve Larry Tesler’e atfedilmektedir; ancak biçimsel bir yayında değil meslek geleneği içinde formüle edilmiştir.

İlişkili bir ayrımı Brooks kurmuştur: yazılımdaki karmaşıklığın bir bölümü problemin kendisinden gelen *özel*, bir bölümü araçlardan ve yöntemlerden gelen *ilineksel* karmaşıklıktır. Araçların iyileştirilmesi yalnızca ikincisini azaltabilir; birincisi problemin doğasında bulunduğu için kaldırılamaz (Brooks, 1987).

Finansman kaynağının belirleyiciliği

Büyük ölçekli açık kaynak projeleri çoğunlukla stratejik hamlelerdir. Bir aracın yol haritasını kestirmek isteyen okuyucu, açıklanan teknik gerekçelere değil, projeyi finanse eden kurumun stratejik konumuna bakmalıdır.

Pratik sonucu şudur: finansör stratejisini değiştirdiğinde araç da değişir; kullanıcının senaryosu aynı kalsa bile. Bugün üzerine sistem inşa edilen bir aracın beş yıl sonraki durumunu belirleyen etken, aracın teknik niteliğinden çok finansörün o tarihteki çıkarıdır.

Belirli projelerin ardındaki kurumsal saikler hakkında yaygın yorumlar bulunmakla birlikte, bu saikler kurumlar tarafından açıkça belgelenmemiştir. Söz konusu yorumlar çıkarım düzeyinde kalmakta olup birincil kaynağa dayandırılmadan olgu gibi aktarılmamalıdır.

Öngörülerin kayda geçirilmesi

Takibin işleyip işlemediğinin dürüst biçimde sınanmasının tek yolu, öngörülerin tarih atılarak yazılması ve sonradan denetlenmesidir. Bu yapılmadığında kişi kendisini olduğundan iyi bir öngörücü sanır, çünkü tutan tahminlerini hatırlar, tutmayanları unuttur.

Eğilim deneysel olarak gösterilmiştir. Bir olayın sonucunu öğrenen kişilerin, olay öncesindeki kendi tahminlerini gerçekleştiren sonuca doğru kaydırarak hatırladıklarını Fischhoff saptamıştır (Fischhoff, 1975). Uzun süreli bir çalışmada Tetlock, uzmanların kendi alanlarındaki tahmin başarımının çoğunlukla sanıldığından düşük olduğunu ve tahminlerin sistematik biçimde kaydedilmesinin performansı iyileştirdiğini bulmuştur (Tetlock, 2005).

²ORM (*Object-Relational Mapping*), veritabanı tablolarını programlama dilindeki nesnelerle eşleyen ara katmandır.

³Çöp toplayıcı, kullanılmayan belleği programcı müdahalesi olmadan geri kazanan çalışma zamanı bileşenidir.

Yanlış çıkan öngörüler doğru çıkanlardan daha fazla bilgi taşır; çünkü modelin hangi noktada gerçekliği tutmadığını gösterirler.

Takibin Sınırları

Dikkatin bölünmesi

Yeni bir alana ayrılan dikkat, derinleşmekte olan bir alandan çekilen dikkattir. Bir alanda gerçek yetkinliğe ulaşmak yıllar alır ve bu yılların büyük bölümü yeni bir şey içermez; aynı işin daha derin biçimde yapılmasından ibarettir.

Sürekli takip hâlindeki kişi her zaman bir konunun başında bulunur, hiçbirinin ortasında bulunmaz. Yüksek yetkinlikli uygulayıcıların önemli bir bölümü geniş takipçiler değil, tek bir alanda uzun süre çalışmış kişilerdir. Takip miktarının uygun düzeyi, opsiyon değeri ile derinlik kaybı arasındaki dengeye göre belirlenir ve çoğu kişi için sanıldığından düşüktür.

Kaygı yönetimi ile öğrenmenin ayrılması

Takip davranışının bir bölümü öğrenme değil kaygı yönetimidir. Geri kalma korkusu somut bir histir ve akış hâlindeki içeriğe bakmak bu hissi geçici olarak yatıştırır. Ancak öğrenme üretmez; üstelik kaygıyı besleyerek döngünün sürmesini sağlar.

Ayırt edici ölçüt şudur: okuma sonrasında edinilen içerik mevcut bir yapıya yerleştirilebiliyor mu, yoksa yalnızca rahatlama mı sağlıyor. İkinci durumda yapılan iş takip değil tüketimdir ve birikim üretmez.

Geç kalmanın maliyetinin abartılması

Bir teknolojiyi iki yıl geç öğrenmenin bedeli çoğunlukla birkaç haftalık öğrenme süresidir. Aradaki dönemde araç olgunlaşmış, belgelendirme düzelmiş ve tuzaclar başkaları tarafından bulunmuştur; bu maliyeti geç benimseyen değil erken benimseyenler ödemiştir.

Bu gözlem, maliyet asimetrisi savını sınırlandırır. Erken benimseme yalnızca söz konusu teknoloji kullanıcının gerçek bir kısıtını çözüyorsa gerekçelidir; merak uyandırdığı için değil. Bu iki saik ayrıştırılmadığında, kişinin emeği kendi kısıtlarını değil başkalarının kısıtlarını çözmeye harcanır.

Soyutlamanın Yapısı

Köken ve temel karar

Terimin kökeni işleyişini açıklamaktadır. Latince *abstrahere* fiili “çekip almak” anlamına gelir. Buna göre soyutlama bir ekleme değil, bir **çıkarma** işlemidir. Türkçedeki karşılığı da aynı kökensel mantığı taşır: soymak, dış katmanı ayırıp geriye özü bırakmaktır.

Bu çerçevede belirleyici soru şudur: hangi kısmın atılacak kabuk, hangi kısmın korunacak öz olduğuna nasıl karar verilmektedir. Soyutlama tasarımının bütün zorluğu bu kararda toplanır.

Yanlış kurulmuş soyutlamanın belirtileri

Sızdırma

Bir soyutlama, kullanılabilmesi için altındaki katmanın bilinmesini gerektiriyor- orsa *sızdırıyor* demektir.

ORM katmanı bunun tipik örneğidir. Sorgu dili yazma zorunluluğunu kaldırmak amacıyla kullanılır; ancak başarımlı sorunu ortaya çıktığında ürettiği sorguların incelenmesi gerekir. Bu durumda kullanıcı hem ORM’i hem sorgu dilini bilmek zorunda kalır; soyutlama bilişsel yükü azaltmak yerine artırmıştır.

Olguyu ilk kez açık biçimde Spolsky formüle etmiş ve önemsiz olmayan bütün soyutlamaların bir ölçüde sızdığını ileri sürmüştür (Spolsky, 2002).

Yanlış eklemden kesme

İyi kurulmuş bir soyutlama, sistemin gerçek ayırım çizgilerinden geçer. Yanlış kurulmuş olan ise birlikte değişmesi gereken parçaları ayırır ya da ayrı değişmesi gerekenleri birleştirir.

Belirtisi gözlemlenebilirdir: tek bir işlevin eklenmesi için birbirinden uzak çok sayıda dosyanın değiştirilmesi gerekiyorsa, ya da birbiriyle ilgisiz iki davranış aynı noktadan bozuluyorsa, kesim yeri yanlış seçilmiştir.

Modül sınırlarının işlev dağılımına göre değil, gizlenen tasarım kararlarına göre çizilmesi gerektiğini Parnas göstermiştir (Parnas, 1972). Buna göre bir modülün sınırı, değişmesi muhtemel bir kararı içine alacak biçimde belirlenmelidir.

İstisna üretimi

Yeterli bir soyutlama az sayıda kavramla çok sayıda durumu kapsar. Yetersiz olan ise her yeni durumda ek bir bayrak, parametre ya da özel durum dalı gerektirir.

Arayüzün zamanla şişmesi bağımsız bir kanıttır: modelin, temsil ettiğini iddia ettiği gerçekliği tutmadığını gösterir. Şişme bir bakım eksikliği değil, tasarım hatasının belirtisidir.

Kaçış kapısı ihtiyacı

Kullanıcıların düzenli biçimde soyutlamayı devre dışı bırakarak alt katmana indiği bir nokta varsa, soyutlama o noktada işlevini yerine getirmiyor demektir.

Kaçış kapısının varlığı tek başına kusur değildir; ender durumlar için bilinçli olarak bırakılabilir. Kusur, kapının olağan kullanımın parçası hâline gelmesidir.

İyi kurulmuş soyutlamanın ölçütleri

Kavram sayısının azalması

Unix sisteminin “her şey dosyadır” ilkesinde disk, uçbirim, boru hattı ve ağ soketi aynı dört çağrıyla kullanılır. Kazanç doğrudan ölçülebilir: öğrenilmesi gereken ayrı arayüz sayısı azalmıştır. Sistemin tasarım ilkeleri ve bu birleştirmenin gerekçesi kurucu makalede açıklanmıştır (Ritchie ve Thompson, 1974).

Öngörülmemiş kullanımların kapsanması

İyi kurulmuş bir soyutlamanın ayırt edici işareti, tasarımcısının aklında olmayan kullanımların da çalışmasıdır.

Unix boru hattı, bugünkü kullanımlarının çoğu için tasarlanmamıştır. Arayüz doğru kurulduğu için sonradan yazılan programlar da aynı zincire eklenebilmektedir; zincire eklenecek programın önceden bilinmesi gerekmez. Yetersiz kurulmuş bir soyutlama ise yalnızca tasarımcının öngördüğü senaryolarda işler.

Atılan bilginin geri istenmemesi

TCP protokolü paket kaybını, sıra bozulmasını ve yeniden gönderimi uygulama katmanından gizler. Sıradan bir uygulama yazılırken bu ayrıntılara ihtiyaç duyulmaz; protokolün uzun ömürlü olmasının nedeni budur. Protokolün tasarımı ve gizlediği sorunlar kurucu makalede tanımlanmıştır (Cerf ve Kahn, 1974).

Değişimin sınırlanması

Doğru çizilmiş bir sınır, alt katmandaki değişikliğin üst katmana yansımamasını sağlar. Aygıt sürücüsü değiştiği hâlde uygulama kodunun değişmemesi, soyutlamanın işlevini yerine getirdiğinin göstergesidir.

Soyutlama Üzerine Kavramsal Çözümleme

“Doğru soyutlama” ifadesinin yanıltıcılığı

Soyutlama bir önerme değildir; dolayısıyla doğruluk ya da yanlışlık yüklemi ona doğrudan uygulanamaz. Daha isabetli terim **uygunluktur** ve bir kesim yerinin isabeti anlamındadır.

Düşüncenin klasik ifadesi Platon’a aittir: iyi bölme, doğayı kendi eklem yerlerinden ayırmaktır (Platon, *Phaidros*, 265e). Benzetmenin mesleki karşılığı açıktır: usta kasap et kaybetmez, çünkü hayvanın nerede eklemli olduğunu bilir; acemi kasap kemiğin ortasından geçer ve her iki tarafta da kullanılamayan artık bırakır.

Aranan ayrım çizgisi tasarımcının çizdiği değil, sistemin kendisinde bulunan çizgidir. Tasarımcının işi çizgiyi icat etmek değil bulmaktır.

Yanlışlığın kaynağı

Yetersiz bir soyutlama, gerçeklikten uzaklaştığı için değil, yanlış bilgiyi attığı için yetersizdir. Soyutlama tanımı gereği bilgi atar; sorun atma işleminin kendisinde değil, atılanın sonradan gerekli olmasındadır.

Katman eğretilemesinin sınırı

Yazılım tartışmasında yaygın olan “katman” eğretilemesi mimariden alınmıştır ve üst üste duran, ağırlığını alta aktaran öğeleri çağırıştırır. Oysa yazılımda alt katman üsttekini taşımaz; üst katman alttakini kullanır. Yapısal ilişki taşıma değil bağımlılıktır.

Daha isabetli bir eğretileme mercektir. Her soyutlama bir odak düzlemi seçer ve bir mesafeyi netleştirmek için diğerlerini bulanıklaştırır.

Ayrımın pratik sonucu vardır. Kat eğretilemesi ekleme işlemini bedelsiz gösterir; bu nedenle “bir katman daha ekleyelim” önerisi kolayca dile getirilir. Mercek eğretilemesinde ekleme bedelsiz değildir, çünkü her mercek bir miktar ışık soğurur.

Sızdırmanın kaza olarak görülmesi

“Sızdıran soyutlama” ifadesi kusuru bir kaza gibi sunar ve yeterince özenli bir tasarımın sızdırmayacağını ima eder.

Daha açıklayıcı bir okuma şudur: her soyutlama, üzerinde uzlaşılmalı ve kasıtlı bir basitleştirmedir. “Bu bir dosyadır” önermesi olgusal olarak doğru değildir; donanım düzeyinde dosya diye bir nesne bulunmaz, yalnızca manyetik ya da elektriksel durumlar vardır.

Bu durumda anlamlı soru basitleştirmenin doğruluğu değil, hangi koşullara kadar dayandığıdır. Hiçbir soyutlama koşulsuz olarak sızdırmaz; bazıları yalnızca kullanıcının hiç girmediği kullanım bölgelerinde sızar.

Sorumluluğun devri

Soyutlamanın kullanıcıyı ayrıntıdan kurtardığı yaygın bir ifadedir, ancak yanıltıcıdır. Ayrıntı ortadan kalkmaz; sahibi değişir.

Çöp toplayıcı bellek yönetimini programcıdan alıp çalışma zamanına devreder. Bellek yönetimi işlemi hâlâ yürütülmektedir ve programcının sorumluluğunda değildir; ancak duraklama süreleri sorun hâline geldiğinde sorumluluk programcıya geri döner.

Buradan genel bir ilke çıkar: her soyutlama bir sorumluluk devridir ve devredilen sorumluluk, sistemin en zorlandığı anda geri gelir. Modüllerin derinliği ile arayüz genişliği arasındaki ilişki üzerine ayrıntılı bir çözümlemeyi Ousterhout yapmıştır (Ousterhout, 2018).

Uygun soyutlamanın önceden bulunamaması

Uygun bir soyutlama, tasarım anında tündengelimle türetilemez; çok sayıda gerçek kullanım gözlemlendikten sonra ortaya çıkar. Bunun nedeni, sistemin hangi eksenlerde değişkenlik gösterdiğinin ancak sistem farklı yönlerde zorlandığında öğrenilebilmesidir.

Bu nedenle kalıcı olmuş soyutlamaların büyük bölümü önceden tasarlanmış değil, çalışan sistemlerden damıtılmıştır. Erken yapılan soyutlama çoğunlukla yanlış çıkar, çünkü hangi eksenin değişken olduğunu belirleyecek gözlem henüz birikmemiştir.

Kurulmuş bir soyutlamanın sonradan düzeltilmesinin güçlüğü ayrıca kullanıcı sayısına bağlıdır. Yeterli sayıda kullanıcıya ulaşan bir arayüzde, belirlenmiş olsun olmasın gözlemlenebilir her davranışın bir kullanıcı tarafından bağımlılık hâline getirileceği gözlemi Hyrum Wright'a atfedilmektedir; formülasyon biçimsel bir yayında değil meslek geleneği içinde dolaşımdadır.

İki Kavramın Kesişimi

Platform yazılımının kopyalanamamasının nedeni

Bir platformu değerli kılan şey kaynak kodu değil, üzerine kurulduğu soyutlamanın isabetidir. Kaynak kodu çoğunlukla açıktır ve indirilebilir; ancak o soyutlamanın hangi denemeler ve başarısızlıklar sonucunda o biçimi aldığı bilgisi kodda yer almaz. Bu bilgi olmadan bir sonraki soyutlama türetilemez.

Donanım alanında taklit edilecek hedef görünür bir nesnedir ve sökülerek incelenebilir. Soyutlama alanında ise hedef, henüz kimse tarafından dile getirilmemiş bir ayırım çizgisidir; ortada sökülecek bir nesne bulunmaz.

Buna ek olarak, olgunlaşmış bir platformun değerinin önemli bölümü kodun kendisinde değil, çevresinde birikmiş katılımcı ağında ve üzerine inşa edilmiş bağımlılıklarda toplanır. Bu birikimin kopyalanabilir bir karşılığı yoktur.

Bu nedenle donanım alanında işleyen yakalama stratejisi platform yazılımında işlememektedir: hızlı ve disiplinli özümseme yeteneği, uygulanacak somut bir yüzey bulamamaktadır.

SpaceX örneğinde soyutlama değişimi

Sarf malzemesi ile araç arasındaki ayırım

Geleneksel fırlatma sistemi tasarımı roketi *sarf malzemesi* olarak soyutlamak-taydı. Bu tek varsayım alt kararların tamamını belirler: her uçuş biricik olduğu için her birim el işçiliğiyle üretilir, birim maliyet yükselir, yüksek maliyet nedeniyle başarısızlık kabul edilemez hâle gelir, bu da doğrulama süreçlerini ağırlaştırır ve maliyeti daha da yükseltir. Ortaya kendini besleyen kapalı bir döngü çıkar.

SpaceX aynı nesneyi *araç* olarak soyutlamıştır. Sorulan soru değişmiştir: “bu roket görevi tamamlar mı” yerine “bu roket kaç kez uçar ve iki uçuş arasında ne kadar iş gerekir” sorusu geçmiştir. Dönüş süresi, muayene yükü ve uçuş başına marjinal maliyet gibi ölçütler, önceki soyutlama altında anlamlı sorular bile değildi.

Hata maliyetinin düşürülmesi

İkinci değişiklik, hata sayısını azaltmak yerine hatanın maliyetini düşürmektir. Bu yaklaşımda test, tasarımın doğrulanması olmaktan çıkıp tasarım sürecinin kendisi hâline gelir; çözümlemeyle elde edilemeyecek bilgi uçuş yoluyla toplanır.

Rejimin ön koşulu tektir: birim maliyetin düşürülmesi. Pahalı olan hiçbir şey çok sayıda denenemez; dolayısıyla maliyet düşürülmeden deneme sayısı artırılmaz. Bu, yazılım geliştirmede olağan olan yüksek yinleme sayısının donanım alanına aktarılmasıdır.

Dikey bütünleşmenin gerekçesi

Üçüncü değişiklik, üretimin büyük bölümünün şirket içinde tutulmasıdır. Gerekçe maliyet değil yinleme hızıdır.

Yeniden kullanım hedefi bütün alt sistemlerin tasarım kısıtlarını aynı anda değiştirir. Çok sayıda dış tedarikçiyle çalışıldığında her değişiklik bir sözleşme müzakeresine dönüşür ve döngü süresi uzar.

Karmaşıklık burada ortadan kaldırılmamış, tedarik zincirinden şirket içine taşınmıştır. Bu, karmaşıklığın korunumu ilkesinin uygulanmış hâlidir: karmaşıklık yok edilemediğine göre, en hızlı yinelenen konuma yerleştirilmiştir.

Biricik parçanın çok sayıda ucuz parçayla değiştirilmesi

Dördüncü değişiklik, büyük ve biricik bir bileşen yerine çok sayıda küçük bileşen kullanılmasıdır. Falcon 9'un birinci kademesinde dokuz, Super Heavy'de otuz üç motor bulunmaktadır.

Bunun iki sonucu vardır. Motor artık düşük adetli özel üretim değil, seri üretim parçasıdır; üretim adedi arttıkça öğrenme etkisi işler ve birim maliyet düşer. İkinci olarak, tek bir motorun kaybı görevi zorunlu olarak sonlandırmaz; yedeklilik tasarımının doğal sonucu olarak elde edilir.

Taklit edilememesinin nedeni

Falcon 9'un genel tasarımı gizli değildir ve iniş manevrasının fiziği kamuya açıktır. Taklit edilemeyen şey, bu soyutlamayı işletebilen kurumsal yapıdır: hızlı yinelemeye izin veren karar mekanizması, başarısızlığı itibar kaybı değil veri kaynağı sayan değerlendirme ölçütü ve tekrarlanan uçuşlarla biriken süreç bilgisi.

Nesne gözlemlenebilir, döngü hızı gözlemlenemez; çünkü ikincisi bir nesne değil bir kurumsal özelliktir.

Silme kuralı

Kuralın içeriği

Musk'ın tasarım süreci olarak aktardığı yöntemde, gereksinimlerin sorgulanmasının ardından parçaların ve süreç adımlarının silinmesi gelir. Kurala göre silinenlerin belirli bir oranı sonradan geri eklenmek zorunda kalmıyorsa, silme işlemi yeterince ileri götürülmemiş demektir.

Buradaki oran bir hedef değil, silmenin yeterliliğini ölçen bir **ayar sinyali**dir. Hiç geri ekleme yapılmamış olması, silme işleminin fazla temkinli yapıldığının kanıtı sayılır.

Yöntemi Musk tesis gezisinde sözlü olarak aktarmış (Everyday Astronaut, 2021), Isaacson ise biyografide beş adımlı bir süreç olarak derlemiştir (Isaacson, 2023). Oranın tam değeri farklı aktarımlarda değişebildiğinden burada oran yerine işlev vurgulanmaktadır.

Ortak epistemik zemin

Soyutlama da bir silme işlemidir ve en güç kısmı neyin atılabileceğini belirlemektir. Kuralın dayandığı bilgi kuramsal saptama şudur: bir bileşenin gerekli olup

olmadığı, çıkarılmadan önce güvenilir biçimde bilinemez.

Tahmin bu durumda sistematik olarak fazla tutmaya yönelir, çünkü iki hatanın maliyetleri asimetriktir. Gereksiz bir parçayı elde tutmanın maliyeti dağıtık ve görünmezdir; gerekli bir parçayı atmanın maliyeti doğrudan ve görünürdür. Karar veren taraf ikinci hatadan kaçınmak için birinciye kabul eder.

Kural bu belirsizliği bir yönteme dönüştürür: sınırın yeri ancak sınır aşılaraq ve bir adım geri dönölerek saptanabilir.

Bu çerçevede sızdıran soyutlama fazla atmış, şişmiş soyutlama az atmış olandır. İkisi de aynı ölçme probleminin karşıt uçlarıdır.

Aktarımın sınırı

Kuralın yazılım soyutlamalarına doğrudan aktarılmasını engelleyen bir fark bulunmaktadır.

Fiziksel parçalar ve süreç adımları birbirinden büyük ölçüde bağımsız olarak çıkarılıp geri konabilir; her deneme yerel ve geri alınabilir. Soyutlama tasarımında yineleme aynı ölçüde ucuz değildir, çünkü yanlış seçilmiş bir kesim yeri yalnızca kendisini değil üzerine kurulmuş her şeyi etkiler. Bir arayüz yanlış çizildiğinde düzeltme maliyeti kullanıcı sayısıyla çarpılır.

Dolayısıyla “sil ve bir bölümünü geri ekle” yöntemi, soyutlama alanında ancak henüz üzerine bağımlılık kurulmamışken uygulanabilir. Bağımlılıklar yerleştikten sonra geri ekleme seçeneği ortadan kalkar; geriye yalnızca uyumluluk katmanı eklemek kahr.

Sonuç

Ortak yapı

Metinde ele alınan üç alan — teknoloji takibi, soyutlama tasarımı ve fiziksel sistem tasarımı — aynı yapısal problemi paylaşır: neyin tutulup neyin atılacağına, sonucu görülmeden karar verilmesi gerekmektedir.

Her üç alanda da fazlalık varsayılan durumdur. Sistemler kendiliğinden sadeleşmez, kendiliğinden şişer. Bunun kurumsal nedeni, ekleme kararını veren ile bedelini ödeyenin çoğunlukla aynı taraf olmamasıdır: ekleme kararı yerel ve görünür bir fayda üretir, bedeli ise sistemin bütününe dağılır ve karar verene fatura edilmez.

Aynı asimetri takip davranışında da işler. Geç kalmanın maliyeti dağıtık olduğu için hissedilmez; erken benimsemenin maliyeti kişiselleştiği için hissedilir. Sonuç, sistematik bir gecikmedir.

Pratik karşılıklar

Yukarıdaki yapıdan üç işlemsel sonuç çıkar.

Birincisi, azaltma kasıtlı bir çaba gerektirir ve kendiliğinden gerçekleşmez. Silme kuralı bunu fiziksel tasarım düzeyinde bir yöntemle bağlar; soyutlama arayışı aynı basıncı kavramsal düzeyde uygular.

İkincisi, sınırın yeri ancak aşılarak bulunabilir. Bir bileşenin gerekliliği çıkarılmadan bilinemediğine göre, çıkarma işlemi bir arama prosedürü olarak kurulmalıdır.

Üçüncüsü, öngörülerin yazılı ve tarihli tutulması, hem takip pratiğinin hem tasarım kararlarının tek dürüst sınama aracıdır. Kayıt tutulmadığında geriye dönük hatırlama sistematik olarak lehte çalışır.

Sınırlılıklar

Çerçevenin geçerlilik alanı sınırlıdır.

Silme kuralı ile soyutlama tasarımı arasındaki benzerlik, yineleme maliyetinin eşit olduğu varsayımına dayanır ve bu varsayım geçerli değildir. Fiziksel parçalarda deneme yerel ve geri alınabilirken, kurulmuş bir arayüzde geri dönüş maliyeti kullanıcı sayısı ile çarpılır. Kural bu nedenle yalnızca bağımlılık birikmemiş soyutlamalara uygulanabilir.

Takip ile ilgili öneriler de koşulsuz değildir. Geç kalmanın maliyeti çoğu durumda abartılmakta olup, erken benimseme yalnızca somut bir kısıt söz konusuysa gerekçelidir. Takip miktarının uygun düzeyi, kişinin çalıştığı alanın değişim hızına bağlıdır ve genel bir sayı verilemez.

Finansman kaynağı ile ürün yönelimi arasındaki bağlantı çıkarım düzeyindedir; kurumsal saikler nadiren belgelendiğinden bu bağlantı öngörü aracı olarak kullanılabılır ancak olgu olarak aktarılamaz.

Açık kalan sorular

Uygun soyutlamanın çalışan sistemlerden damıtıldığı savı, damıtmanın hangi noktada yapılacağını belirlemez. Erken soyutlamanın yanlış, geç soyutlamanın maliyetli olduğu kabul edildiğinde, aradaki eşğin nasıl saptanacağı açık kalmaktadır.

Karmaşıklığın korunumu ilkesi niteliksel bir gözlemdir; karmaşıklığın hangi birimle ölçüleceği ve taşındığı konumdaki artışın kaldırıldığı konumdaki azalmayla karşılaştırılıp karşılaştırılmayacağı belirsizdir.

Kurumsal düzeyde, görünmez maliyetin görünür kılınmasını sağlayacak mekanizmaların neler olabileceği bu metinde ele alınmamıştır. Sorunun kaynağı muhasebe biçimi olduğuna göre çözümün de orada aranması beklenir.

Kaynaklar

- Brooks, F. P. (1987). No Silver Bullet: Essence and Accidents of Software Engineering. *IEEE Computer*, 20(4).
- Cerf, V. ve Kahn, R. (1974). A Protocol for Packet Network Intercommunication. *IEEE Transactions on Communications*, 22(5).
- Everyday Astronaut (2021). Starbase Tour with Elon Musk. Video söyleşi.
- Fischhoff, B. (1975). Hindsight $\neq$ Foresight: The Effect of Outcome Knowledge on Judgment Under Uncertainty. *Journal of Experimental Psychology: Human Perception and Performance*, 1(3).
- Isaacson, W. (2023). *Elon Musk*. Simon & Schuster.
- Ousterhout, J. (2018). *A Philosophy of Software Design*. Yaknyam Press.
- Parnas, D. L. (1972). On the Criteria To Be Used in Decomposing Systems into Modules. *Communications of the ACM*, 15(12).
- Platon. *Phaidros*, 265e.
- Ritchie, D. M. ve Thompson, K. (1974). The UNIX Time-Sharing System. *Communications of the ACM*, 17(7).
- Rogers, E. M. (1962). *Diffusion of Innovations*. Free Press.
- Spolsky, J. (2002). The Law of Leaky Abstractions. *Joel on Software*.
- Tetlock, P. E. (2005). *Expert Political Judgment: How Good Is It? How Can We Know?*. Princeton University Press.